Threagile
Agile Threat Modeling

Threat Model Report
VaultNote Threat Model

18 April 2026

VaultNote Security Team

https://github.com/vaultnote

Table of Contents

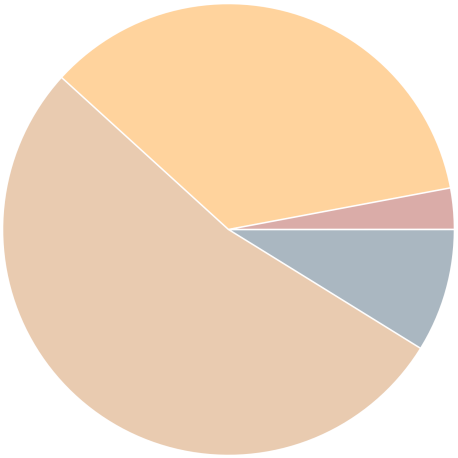
Management Summary

Threagile toolkit was used to model the architecture of "VaultNote Threat Model" and derive risks by analyzing the components and data flows. The risks identified during this analysis are shown in the following chapters. Identified risks during threat modeling do not necessarily mean that the vulnerability associated with this risk actually exists: it is more to be seen as a list of potential risks and threats, which should be individually reviewed and reduced by removing false positives. For the remaining risks it should be checked in the design and implementation of "VaultNote Threat Model" whether the mitigation advices have been applied or not.

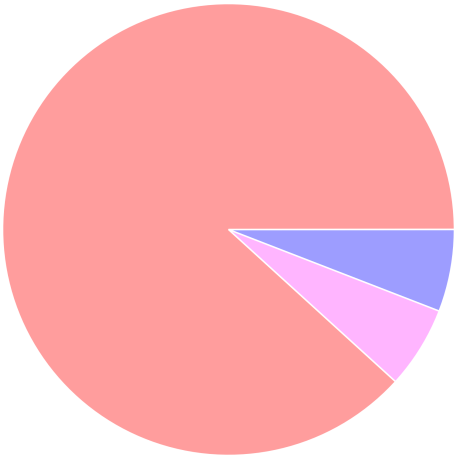
Each risk finding references a chapter of the OWASP ASVS (Application Security Verification Standard) audit checklist. The OWASP ASVS checklist should be considered as an inspiration by architects and developers to further harden the application in a Defense-in-Depth approach. Additionally, for each risk finding a link towards a matching OWASP Cheat Sheet or similar with technical details about how to implement a mitigation is given.

In total **34 initial risks** in **16 categories** have been identified during the threat modeling process:

0 critical risk
1 high risk
12 elevated risk
18 medium risk
3 low risk



30 unchecked
0 in discussion
2 accepted
2 in progress
0 *mitigated*
0 *false positive*



VaultNote is a containerized note-taking application comprised of a React SPA, Nginx reverse proxy, Node.js/Express REST API, PostgreSQL database, Redis session store, and MinIO object storage. The architecture deliberately contains several intentional misconfigurations (unencrypted internal HTTP, no encryption at rest, hardcoded credentials) to demonstrate automated threat detection via Threagile. All findings below are expected and serve as demonstration targets.

Impact Analysis of 34 Initial Risks in 16 Categories

The most prevalent impacts of the **34 initial risks** (distributed over **16 risk categories**) are (taking the severity ratings into account and using the highest for each category):
Risk finding paragraphs are clickable and link to the corresponding chapter.

High: **SQL/NoSQL-Injection**: 2 Initial Risks - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: **Missing Authentication**: 1 Initial Risk - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: **Missing Hardening**: 2 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Elevated: **Path-Traversal**: 1 Initial Risk - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

Elevated: **Server-Side Request Forgery (SSRF)**: 2 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Elevated: **Unencrypted Communication**: 4 Initial Risks - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Elevated: **Unguarded Direct Datastore Access**: 3 Initial Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive datastores without any protecting components in-between.

Medium: **Container Base Image Backdooring**: 5 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Medium: **Missing Build Infrastructure**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: **Missing Identity Store**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model in the identity provider/store that is currently missing in the model.

Medium: **Missing Two-Factor Authentication (2FA)**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Medium: **Missing Vault (Secret Storage)**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: **Missing Web Application Firewall (WAF)**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

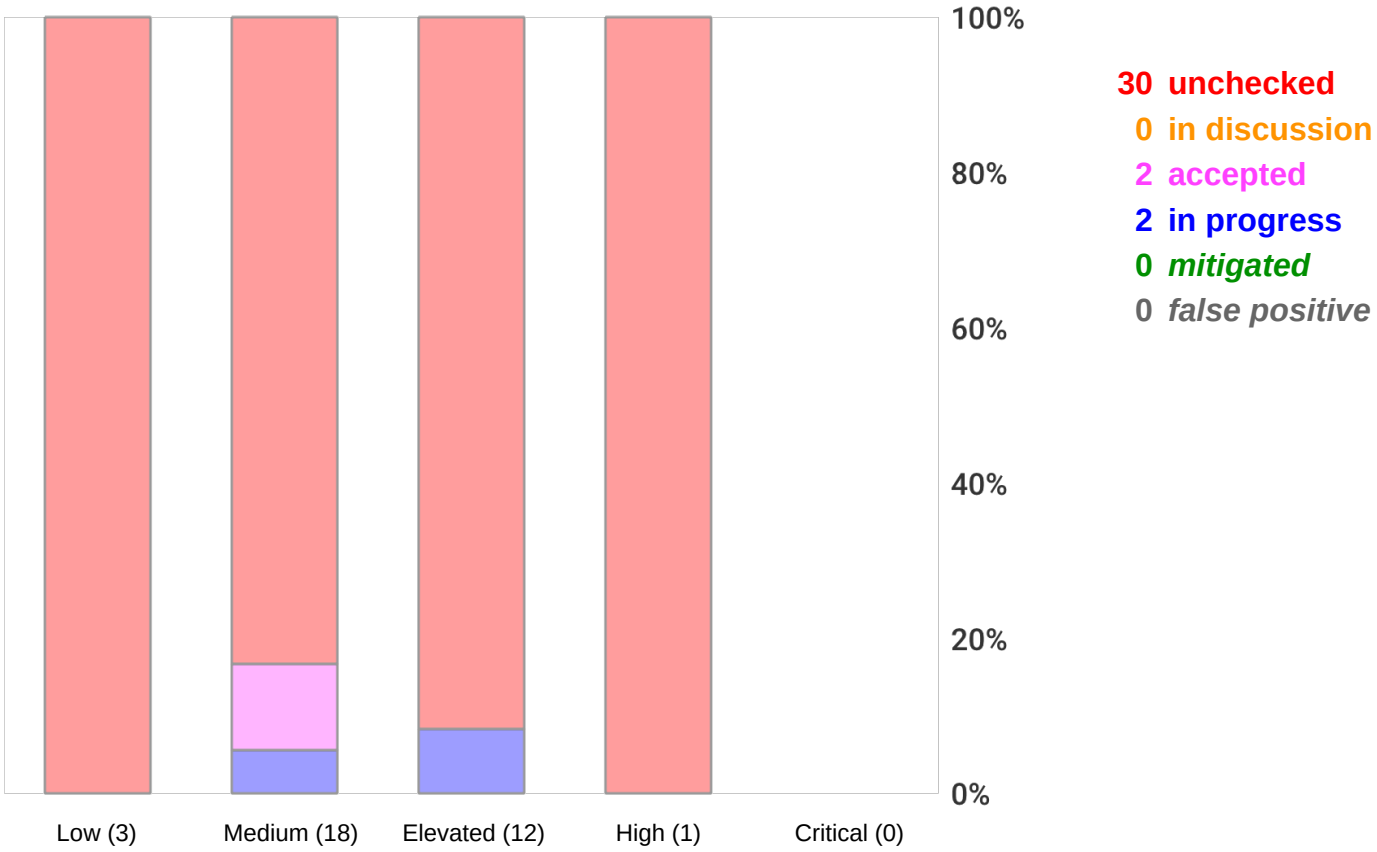
Medium: **Mixed Targets on Shared Runtime**: 1 Initial Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are running on the same shared runtime.

Medium: **Unencrypted Technical Assets**: 5 Initial Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Low: **DoS-risky Access Across Trust-Boundary**: 3 Initial Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
If this risk remains unmitigated, attackers might be able to disturb the availability of important parts of the system.

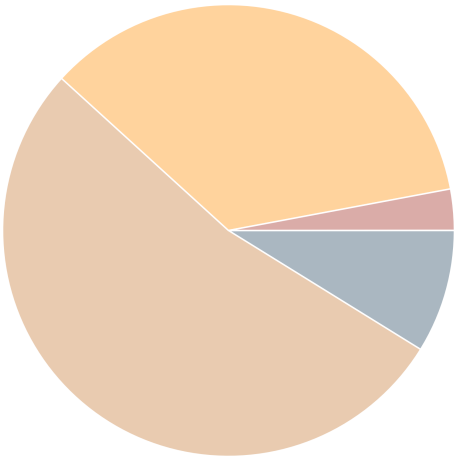
Risk Mitigation

The following chart gives a high-level overview of the risk tracking status (including mitigated risks):

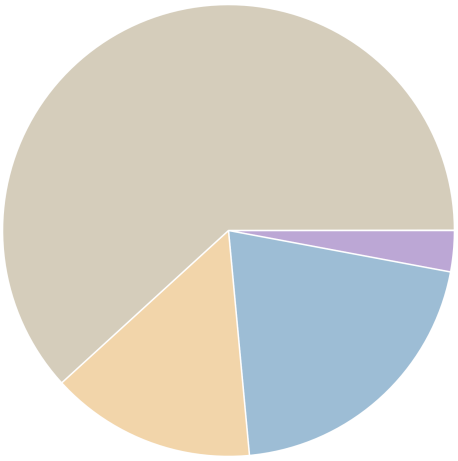


After removal of risks with status *mitigated* and *false positive* the following **34 remain unmitigated**:

0 unmitigated critical risk
1 unmitigated high risk
12 unmitigated elevated risk
18 unmitigated medium risk
3 unmitigated low risk



1 business side related
7 architecture related
5 development related
21 operations related



Impact Analysis of 34 Remaining Risks in 16 Categories

The most prevalent impacts of the **34 remaining risks** (distributed over **16 risk categories**) are (taking the severity ratings into account and using the highest for each category):
Risk finding paragraphs are clickable and link to the corresponding chapter.

High: **SQL/NoSQL-Injection**: 2 Remaining Risks - Exploitation likelihood is *Very Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Elevated: **Missing Authentication**: 1 Remaining Risk - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Elevated: **Missing Hardening**: 2 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Elevated: **Path-Traversal**: 1 Remaining Risk - Exploitation likelihood is *Very Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

Elevated: **Server-Side Request Forgery (SSRF)**: 2 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Elevated: **Unencrypted Communication**: 4 Remaining Risks - Exploitation likelihood is *Likely* with *High* impact.
If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Elevated: **Unguarded Direct Datastore Access**: 3 Remaining Risks - Exploitation likelihood is *Likely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to directly attack sensitive datastores without any protecting components in-between.

Medium: **Container Base Image Backdooring**: 5 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Medium: **Missing Build Infrastructure**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Medium: **Missing Identity Store**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model in the identity provider/store that is currently missing in the model.

Medium: **Missing Two-Factor Authentication (2FA)**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Medium: **Missing Vault (Secret Storage)**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Medium: **Missing Web Application Firewall (WAF)**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Medium: **Mixed Targets on Shared Runtime**: 1 Remaining Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are running on the same shared runtime.

Medium: **Unencrypted Technical Assets**: 5 Remaining Risks - Exploitation likelihood is *Unlikely* with *High* impact.
If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Low: **DoS-risky Access Across Trust-Boundary**: 3 Remaining Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
If this risk remains unmitigated, attackers might be able to disturb the availability of important parts of the system.

Application Overview

Business Criticality

The overall business criticality of "VaultNote Threat Model" was rated as:

(archive | operational | **IMPORTANT** | critical | mission-critical)

Business Overview

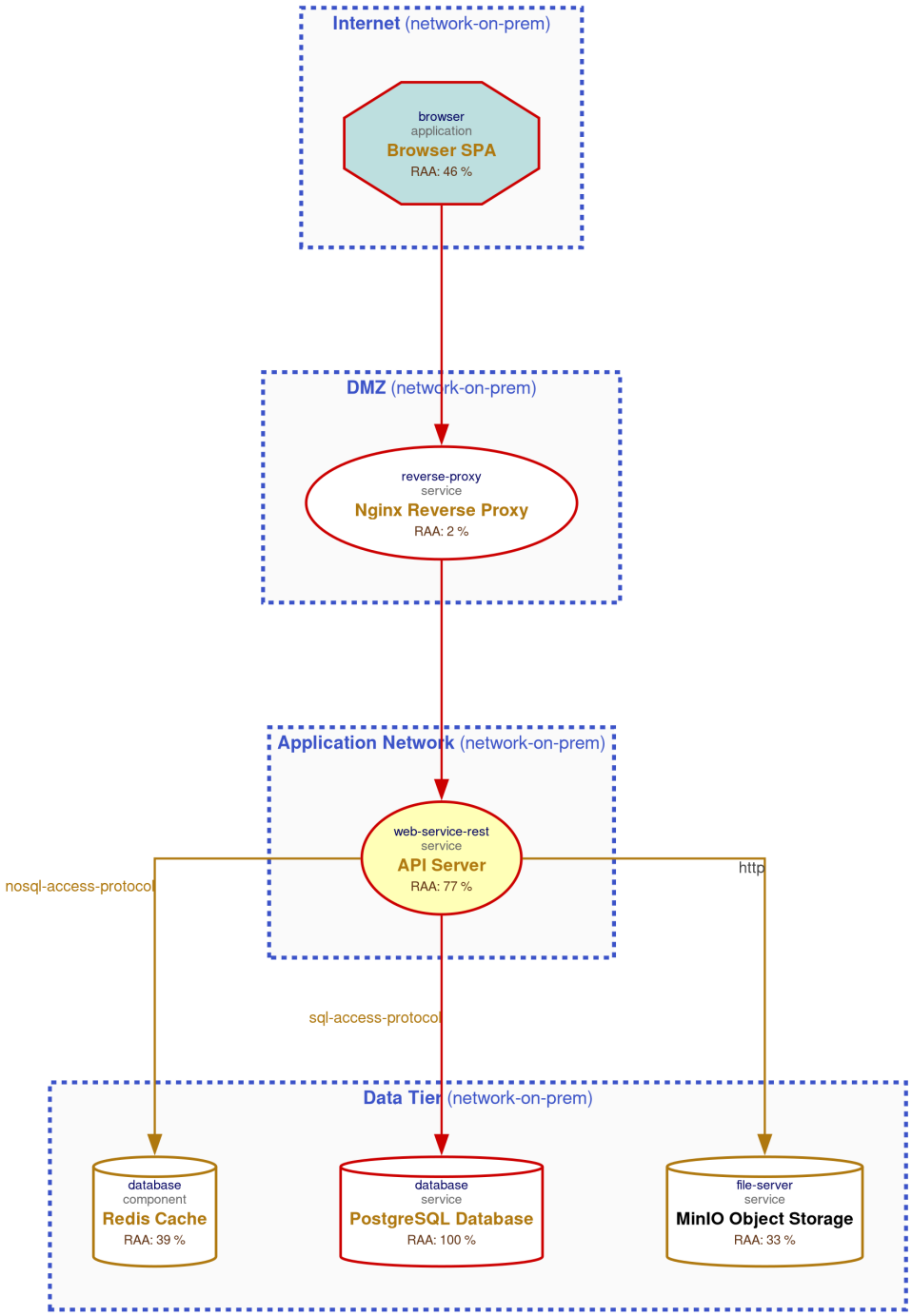
VaultNote allows authenticated users to create, edit, and share encrypted notes with file attachments. The application stores sensitive user data and note content that must be protected against unauthorized disclosure.

Technical Overview

Multi-tier containerized deployment: Nginx terminates TLS and proxies requests to the API over unencrypted HTTP. The API accesses PostgreSQL, Redis, and MinIO over plaintext TCP/HTTP within an internal Docker network.

Data-Flow Diagram

The following diagram was generated by Threagile based on the model input and gives a high-level overview of the data-flow between technical assets. The RAA value is the calculated *Relative Attacker Attractiveness* in percent. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Security Requirements

This chapter lists the custom security requirements which have been defined for the modeled target.

This list is not complete and regulatory or law relevant security requirements have to be taken into account as well. Also custom individual security requirements might exist for the project.

Abuse Cases

This chapter lists the custom abuse cases which have been defined for the modeled target.

This list is not complete and regulatory or law relevant abuse cases have to be taken into account as well. Also custom individual abuse cases might exist for the project.

Tag Listing

This chapter lists what tags are used by which elements.

cache
Redis Cache

container-runtime
Docker Host

docker
Docker Host

express
API Server

intentional-misconfiguration
MinIO File Storage, HTTP to API Server

jwt
API Server

minio
MinIO Object Storage

nginx
Nginx Reverse Proxy

nodejs
API Server

object-storage
MinIO Object Storage

postgresql
PostgreSQL Database

react
Browser SPA

redis
Redis Cache

relational
PostgreSQL Database

s3
MinIO Object Storage

session-store
Redis Cache

spa
Browser SPA

tls-termination
Nginx Reverse Proxy

STRIDE Classification of Identified Risks

This chapter clusters and classifies the risks by STRIDE categories: In total **34 potential risks** have been identified during the threat modeling process of which **1 in the Spoofing** category, **11 in the Tampering** category, **0 in the Repudiation** category, **13 in the Information Disclosure** category, **3 in the Denial of Service** category, and **6 in the Elevation of Privilege** category.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Spoofing

Medium: **Missing Identity Store**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Tampering

High: **SQL/NoSQL-Injection**: 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *High* impact.

When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed or stored.

Elevated: **Missing Hardening**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *Medium* impact.

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Medium: **Container Base Image Backdooring**: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.

When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: **Missing Web Application Firewall (WAF)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Repudiation

n/a

Information Disclosure

Elevated: **Path-Traversal**: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *Medium* impact.
When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed or stored.

Elevated: **Server-Side Request Forgery (SSRF)**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Elevated: **Unencrypted Communication**: 4 / 4 Risks - Exploitation likelihood is *Likely* with *High* impact.
Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Medium: **Unencrypted Technical Assets**: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
Due to the confidentiality rating of the technical asset itself and/or the processed data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Denial of Service

Low: **DoS-risky Access Across Trust-Boundary**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
Assets accessed across trust boundaries with critical or mission-critical availability rating are more prone to Denial-of-Service (DoS) risks.

Elevation of Privilege

Elevated: **Missing Authentication**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *High* impact.
Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes or stores sensitive data.

Elevated: **Unguarded Direct Datastore Access**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Datastores accessed across trust boundaries must be guarded by some protecting service or application.

Medium: **Missing Two-Factor Authentication (2FA)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Medium: **Mixed Targets on Shared Runtime**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Different attacker targets (like frontend and backend/datastore components) should not be running on the same shared (underlying) runtime.

Assignment by Function

This chapter clusters and assigns the risks by functions which are most likely able to check and mitigate them: In total **34 potential risks** have been identified during the threat modeling process of which **1 should be checked by Business Side**, **7 should be checked by Architecture**, **5 should be checked by Development**, and **21 should be checked by Operations**.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Business Side

Medium: **Missing Two-Factor Authentication (2FA)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Apply an authentication method to the technical asset protecting highly sensitive data via two-factor authentication for human users.

Architecture

Elevated: **Missing Authentication**: 1 / 1 Risk - Exploitation likelihood is *Likely* with *High* impact.
Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

Elevated: **Unguarded Direct Datastore Access**: 3 / 3 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Encapsulate the datastore access behind a guarding service or application.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Include the build infrastructure in the model.

Medium: **Missing Identity Store**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Include an identity store in the model if the application has a login.

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

Development

High: **SQL/NoSQL-Injection**: 2 / 2 Risks - Exploitation likelihood is *Very Likely* with *High* impact.
Try to use parameter binding to be safe from injection vulnerabilities. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Elevated: **Path-Traversal**: 1 / 1 Risk - Exploitation likelihood is *Very Likely* with *Medium* impact.
Before accessing the file cross-check that it resides in the expected folder and is of the expected type and filename/suffix. Try to use a mapping if possible instead of directly accessing by a filename which is (partly or fully) provided by the caller. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Elevated: **Server-Side Request Forgery (SSRF)**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

Operations

Elevated: **Missing Hardening**: 2 / 2 Risks - Exploitation likelihood is *Likely* with *Medium* impact.
Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

Elevated: **Unencrypted Communication**: 4 / 4 Risks - Exploitation likelihood is *Likely* with *High* impact.
Apply transport layer encryption to the communication link.

Medium: **Container Base Image Backdooring**: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
Apply hardening of all container infrastructures (see for example the *CIS-Benchmarks for Docker and Kubernetes* and the *Docker Bench for Security*). Use only trusted base images of the original vendors, verify digital signatures and apply image creation best practices. Also consider using Google's *Distroless* base images or otherwise very small base images. Regularly execute container image scans with tools checking the layers for vulnerable components.

Medium: **Missing Web Application Firewall (WAF)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Consider placing a Web Application Firewall (WAF) in front of the web-services and/or web-applications. For cloud environments many cloud providers offer pre-configured WAFs. Even reverse proxies can be enhances by a WAF component via ModSecurity plugins.

Medium: **Mixed Targets on Shared Runtime**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.
Use separate runtime environments for running different target components or apply similar separation styles to prevent load- or breach-related problems originating from one more attacker-facing asset impacts also the other more critical rated backend/datastore assets.

Medium: **Unencrypted Technical Assets**: 5 / 5 Risks - Exploitation likelihood is *Unlikely* with *High* impact.
Apply encryption to the technical asset.

Low: **DoS-risky Access Across Trust-Boundary**: 3 / 3 Risks - Exploitation likelihood is *Unlikely* with *Low* impact.
Apply anti-DoS techniques like throttling and/or per-client load blocking with quotas. Also for maintenance access routes consider applying a VPN instead of public reachable interfaces. Generally applying redundancy on the targeted technical asset reduces the risk of DoS.

RAA Analysis

For each technical asset the **"Relative Attacker Attractiveness"** (RAA) value was calculated in percent. The higher the RAA, the more interesting it is for an attacker to compromise the asset. The calculation algorithm takes the sensitivity ratings and quantities of stored and processed data into account as well as the communication links of the technical asset. Neighbouring assets to high-value RAA targets might receive an increase in their RAA value when they have a communication link towards that target ("Pivoting-Factor").

The following lists all technical assets sorted by their RAA value from highest (most attacker attractive) to lowest. This list can be used to prioritize on efforts relevant for the most attacker-attractive technical assets:

Technical asset paragraphs are clickable and link to the corresponding chapter.

PostgreSQL Database: RAA 100%
Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

API Server: RAA 77%
Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Browser SPA: RAA 46%
React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

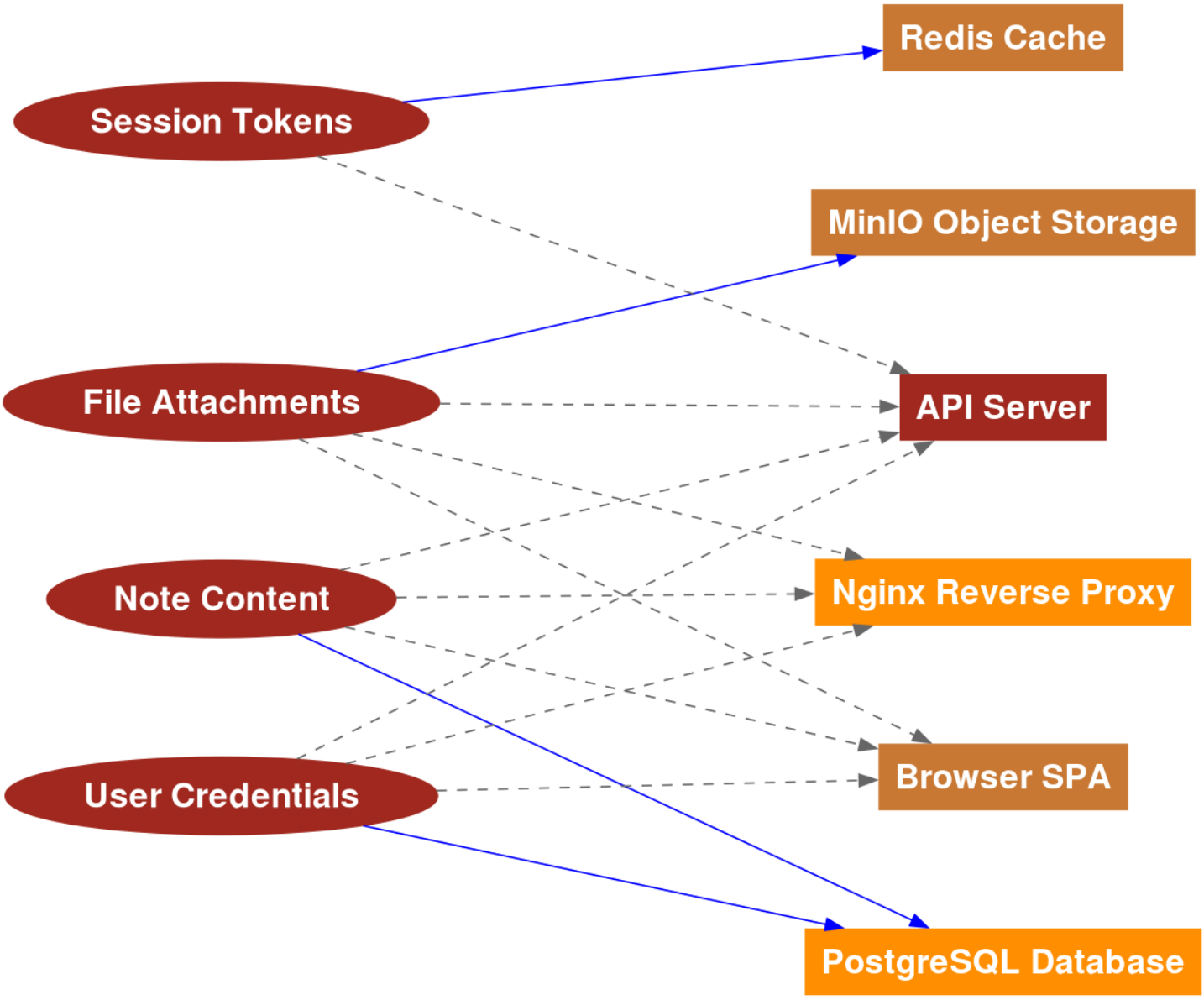
Redis Cache: RAA 39%
In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

MinIO Object Storage: RAA 33%
S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Nginx Reverse Proxy: RAA 2%
Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Data Mapping

The following diagram was generated by Threagile based on the model input and gives a high-level distribution of data assets across technical assets. The color matches the identified data breach probability and risk level (see the "Data Breach Probabilities" chapter for more details). A solid line stands for *data is stored by the asset* and a dashed one means *data is processed by the asset*. For a full high-resolution version of this diagram please refer to the PNG image file alongside this report.



Out-of-Scope Assets: 0 Assets

This chapter lists all technical assets that have been defined as out-of-scope. Each one should be checked in the model whether it should better be included in the overall risk analysis:

Technical asset paragraphs are clickable and link to the corresponding chapter.

No technical assets have been defined as out-of-scope.

Potential Model Failures: 3 / 3 Risks

This chapter lists potential model failures where not all relevant assets have been modeled or the model might itself contain inconsistencies. Each potential model failure should be checked in the model against the architecture design:

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium: **Missing Build Infrastructure**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Medium: **Missing Identity Store**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Medium: **Missing Vault (Secret Storage)**: 1 / 1 Risk - Exploitation likelihood is *Unlikely* with *Medium* impact.

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Questions: 0 / 0 Questions

This chapter lists custom questions that arose during the threat modeling process.

No custom questions arose during the threat modeling process.

Identified Risks by Vulnerability Category

In total **34 potential risks** have been identified during the threat modeling process of which **0 are rated as critical**, **1 as high**, **12 as elevated**, **18 as medium**, and **3 as low**.

These risks are distributed across **16 vulnerability categories**. The following sub-chapters of this section describe each identified risk category.

SQL/NoSQL-Injection: 2 / 2 Risks

Description (Tampering): CWE 89

When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed or stored.

Impact

If this risk is unmitigated, attackers might be able to modify SQL/NoSQL queries to steal and modify data and eventually further escalate towards a deeper system penetration via code executions.

Detection Logic

Database accessed via typical database access protocols by in-scope clients.

Risk Rating

The risk rating depends on the sensitivity of the data stored inside the database.

False Positives

Database accesses by queries not consisting of parts controllable by the caller can be considered as false positives after individual review.

Mitigation (Development): SQL/NoSQL-Injection Prevention

Try to use parameter binding to be safe from injection vulnerabilities. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: V5 - Validation, Sanitization and Encoding Verification Requirements
Cheat Sheet: SQL_Injection_Prevention_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **SQL/NoSQL-Injection** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

SQL/NoSQL-Injection risk at **API Server** against database **PostgreSQL Database** via **PostgreSQL Connection**: Exploitation likelihood is *Very Likely* with *High* impact.

sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Unchecked

Elevated Risk Severity

SQL/NoSQL-Injection risk at **API Server** against database **Redis Cache** via **Redis Session Store**: Exploitation likelihood is *Very Likely* with *Medium* impact.

sql-nosql-injection@api-server@redis-cache@api-server>redis-session-store

Unchecked

Missing Authentication: 1 / 1 Risk

Description (Elevation of Privilege): CWE 306

Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes or stores sensitive data.

Impact

If this risk is unmitigated, attackers might be able to access or modify sensitive data in an unauthenticated way.

Detection Logic

In-scope technical assets (except load-balancer, reverse-proxy, service-registry, waf, ids, and ips and in-process calls) should authenticate incoming requests when the asset processes or stores sensitive data. This is especially the case for all multi-tenant assets (there even non-sensitive ones).

Risk Rating

The risk rating (medium or high) depends on the sensitivity of the data sent across the communication link. Monitoring callers are exempted from this risk.

False Positives

Technical assets which do not process requests regarding functionality or data linked to end-users (customers) can be considered as false positives after individual review.

Mitigation (Architecture): Authentication of Incoming Requests

Apply an authentication method to the technical asset. To protect highly sensitive data consider the use of two-factor authentication for human users.

ASVS Chapter: V2 - Authentication Verification Requirements
Cheat Sheet: Authentication_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Authentication** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Authentication covering communication link **HTTP to API Server** from **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Likely* with *High* impact.

missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Missing Hardening: 2 / 2 Risks

Description (Tampering): CWE 16

Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.

Impact

If this risk remains unmitigated, attackers might be able to easier attack high-value targets.

Detection Logic

In-scope technical assets with RAA values of 55 % or higher. Generally for high-value targets like datastores, application servers, identity providers and ERP systems this limit is reduced to 40 %

Risk Rating

The risk rating depends on the sensitivity of the data processed or stored in the technical asset.

False Positives

Usually no false positives.

Mitigation (Operations): System Hardening

Try to apply all hardening best practices (like CIS benchmarks, OWASP recommendations, vendor recommendations, DevSec Hardening Framework, DBSAT for Oracle databases, and others).

ASVS Chapter: V14 - Configuration Verification Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Hardening** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **API Server**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@api-server

Unchecked

Missing Hardening risk at **PostgreSQL Database**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@postgresql-db

Unchecked

Path-Traversal: 1 / 1 Risk

Description (Information Disclosure): CWE 22

When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed or stored.

Impact

If this risk is unmitigated, attackers might be able to read sensitive files (configuration data, key/credential files, deployment files, business data files, etc.) from the filesystem of affected components.

Detection Logic

Filesystems accessed by in-scope callers.

Risk Rating

The risk rating depends on the sensitivity of the data stored inside the technical asset.

False Positives

File accesses by filenames not consisting of parts controllable by the caller can be considered as false positives after individual review.

Mitigation (Development): Path-Traversal Prevention

Before accessing the file cross-check that it resides in the expected folder and is of the expected type and filename/suffix. Try to use a mapping if possible instead of directly accessing by a filename which is (partly or fully) provided by the caller. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: V12 - File and Resources Verification Requirements
Cheat Sheet: Input_Validation_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Path-Traversal** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Path-Traversal risk at **API Server** against filesystem **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Very Likely* with *Medium* impact.

path-traversal@api-server@minio-storage@api-server>minio-file-storage

Unchecked

Server-Side Request Forgery (SSRF): 2 / 2 Risks

Description (Information Disclosure): CWE 918

When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Impact

If this risk is unmitigated, attackers might be able to access sensitive services or files of network-reachable components by modifying outgoing calls of affected components.

Detection Logic

In-scope non-client systems accessing (using outgoing communication links) targets with either HTTP or HTTPS protocol.

Risk Rating

The risk rating (low or medium) depends on the sensitivity of the data assets receivable via web protocols from targets within the same network trust-boundary as well on the sensitivity of the data assets receivable via web protocols from the target asset itself. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF.

False Positives

Servers not sending outgoing web requests can be considered as false positives after review.

Mitigation (Development): SSRF Prevention

Try to avoid constructing the outgoing target URL with caller controllable values. Alternatively use a mapping (whitelist) when accessing outgoing URLs instead of creating them including caller controllable values. When a third-party product is used instead of custom developed software, check if the product applies the proper mitigation and ensure a reasonable patch-level.

ASVS Chapter: V12 - File and Resources Verification Requirements
Cheat Sheet: Server_Side_Request_Forgery_Prevention_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Server-Side Request Forgery (SSRF)** was found **2 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Server-Side Request Forgery (SSRF) risk at **API Server** server-side web-requesting the target **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Unchecked

Server-Side Request Forgery (SSRF) risk at **Nginx Reverse Proxy** server-side web-requesting the target **API Server** via **HTTP to API Server**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@nginx-proxy@api-server@nginx-proxy>http-to-api-server

Unchecked

Unencrypted Communication: 4 / 4 Risks

Description (Information Disclosure): CWE 319

Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Impact

If this risk is unmitigated, network attackers might be able to to eavesdrop on unencrypted sensitive data sent between components.

Detection Logic

Unencrypted technical communication links of in-scope technical assets (excluding monitoring traffic as well as local-file-access and in-process-library-call) transferring sensitive data.

Risk Rating

Depending on the confidentiality rating of the transferred data-assets either medium or high risk.

False Positives

When all sensitive data sent over the communication link is already fully encrypted on document or data level. Also intra-container/pod communication can be considered false positive when container orchestration platform handles encryption.

Mitigation (Operations): Encryption of Communication Links

Apply transport layer encryption to the communication link.

ASVS Chapter: V9 - Communication Verification Requirements
Cheat Sheet: Transport_Layer_Protection_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unencrypted Communication** was found **4 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unencrypted Communication named **MinIO File Storage** between **API Server** and **MinIO Object Storage** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unencrypted Communication named **PostgreSQL Connection** between **API Server** and **PostgreSQL Database** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Unencrypted Communication named **Redis Session Store** between **API Server** and **Redis Cache** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>redis-session-store@api-server@redis-cache

Unchecked

Unencrypted Communication named **HTTP to API Server** between **Nginx Reverse Proxy** and **API Server**: Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

in Progress 2026-04-18 Security Team VAULT-102

Migrating Nginx-to-API communication to mTLS. A separate CA will be provisioned inside the Docker network. PR #47 is in review; target completion sprint 22.

Unguarded Direct Datastore Access: 3 / 3 Risks

Description (Elevation of Privilege): CWE 501

Datastores accessed across trust boundaries must be guarded by some protecting service or application.

Impact

If this risk is unmitigated, attackers might be able to directly attack sensitive datastores without any protecting components in-between.

Detection Logic

In-scope technical assets of type datastore (except identity-store-ldap when accessed from identity-provider and file-server when accessed via file transfer protocols) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) which have incoming data-flows from assets outside across a network trust-boundary. DevOps config and deployment access is excluded from this risk.

Risk Rating

The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

False Positives

When the caller is considered fully trusted as if it was part of the datastore itself.

Mitigation (Architecture): Encapsulation of Datastore

Encapsulate the datastore access behind a guarding service or application.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unguarded Direct Datastore Access** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Unguarded Direct Datastore Access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Likely* with *Medium* impact.

unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Medium Risk Severity

Unguarded Direct Datastore Access of **MinIO Object Storage** by **API Server** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unguarded Direct Datastore Access of **Redis Cache** by **API Server** via **Redis Session Store**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>redis-session-store@api-server@redis-cache

Unchecked

Container Base Image Backdooring: 5 / 5 Risks

Description (Tampering): CWE 912

When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.

See for example:
https://techcrunch.com/2018/06/15/tainted-crypto-mining-containers-pulled-from-docker-hub/

Impact

If this risk is unmitigated, attackers might be able to deeply persist in the target system by executing code in deployed containers.

Detection Logic

In-scope technical assets running as containers.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets.

False Positives

Fully trusted (i.e. reviewed and cryptographically signed or similar) base images of containers can be considered as false positives after individual review.

Mitigation (Operations): Container Infrastructure Hardening

Apply hardening of all container infrastructures (see for example the *CIS-Benchmarks for Docker and Kubernetes* and the *Docker Bench for Security*). Use only trusted base images of the original vendors, verify digital signatures and apply image creation best practices. Also consider using Google's *Distroless* base images or otherwise very small base images. Regularly execute container image scans with tools checking the layers for vulnerable components.

ASVS Chapter: V10 - Malicious Code Verification Requirements
Cheat Sheet: Docker_Security_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS/CSVS applied?

Risk Findings

The risk **Container Base Image Backdooring** was found **5 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **API Server**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@api-server

Unchecked

Container Base Image Backdooring risk at **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@nginx-proxy

Unchecked

Container Base Image Backdooring risk at **PostgreSQL Database**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@postgresql-db

Unchecked

Container Base Image Backdooring risk at **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@minio-storage

Unchecked

Container Base Image Backdooring risk at **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@redis-cache

Unchecked

Missing Build Infrastructure: 1 / 1 Risk

Description (Tampering): CWE 1127

The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Impact

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model due to critical build infrastructure components missing in the model.

Detection Logic

Models with in-scope custom-developed parts missing in-scope development (code creation) and build infrastructure components (devops-client, sourcecode-repo, build-pipeline, etc.).

Risk Rating

The risk rating depends on the highest sensitivity of the in-scope assets running custom-developed parts.

False Positives

Models not having any custom-developed parts can be considered as false positives after individual review.

Mitigation (Architecture): Build Pipeline Hardening

Include the build infrastructure in the model.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Build Infrastructure** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Build Infrastructure in the threat model (referencing asset **API Server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-build-infrastructure@api-server

Unchecked

Missing Identity Store: 1 / 1 Risk

Description (Spoofing): CWE 287

The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Impact

If this risk is unmitigated, attackers might be able to exploit risks unseen in this threat model in the identity provider/store that is currently missing in the model.

Detection Logic

Models with authenticated data-flows authorized via enduser-identity missing an in-scope identity store.

Risk Rating

The risk rating depends on the sensitivity of the enduser-identity authorized technical assets and their data assets processed and stored.

False Positives

Models only offering data/services without any real authentication need can be considered as false positives after individual review.

Mitigation (Architecture): Identity Store

Include an identity store in the model if the application has a login.

ASVS Chapter: V2 - Authentication Verification Requirements
Cheat Sheet: Authentication_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Identity Store** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Identity Store in the threat model (referencing asset **Nginx Reverse Proxy** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-identity-store@nginx-proxy

Unchecked

Missing Two-Factor Authentication (2FA): 1 / 1 Risk

Description (Elevation of Privilege): CWE 308

Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Impact

If this risk is unmitigated, attackers might be able to access or modify highly sensitive data without strong authentication.

Detection Logic

In-scope technical assets (except load-balancer, reverse-proxy, waf, ids, and ips) should authenticate incoming requests via two-factor authentication (2FA) when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by a client used by a human user.

Risk Rating

medium

False Positives

Technical assets which do not process requests regarding functionality or data linked to end-users (customers) can be considered as false positives after individual review.

Mitigation (Business Side): Authentication with Second Factor (2FA)

Apply an authentication method to the technical asset protecting highly sensitive data via two-factor authentication for human users.

ASVS Chapter: V2 - Authentication Verification Requirements
Cheat Sheet: Multifactor_Authentication_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Missing Two-Factor Authentication (2FA)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Two-Factor Authentication covering communication link **HTTP to API Server** from **Browser SPA** forwarded via **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Missing Vault (Secret Storage): 1 / 1 Risk

Description (Information Disclosure): CWE 522

In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Impact

If this risk is unmitigated, attackers might be able to easier steal config secrets (like credentials, private keys, client certificates, etc.) once a vulnerability to access files is present and exploited.

Detection Logic

Models without a Vault (Secret Storage).

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

False Positives

Models where no technical assets have any kind of sensitive config data to protect can be considered as false positives after individual review.

Mitigation (Architecture): Vault (Secret Storage)

Consider using a Vault (Secret Storage) to securely store and access config secrets (like credentials, private keys, client certificates, etc.).

ASVS Chapter: V6 - Stored Cryptography Verification Requirements
Cheat Sheet: Cryptographic_Storage_Cheat_Sheet

Check

Is a Vault (Secret Storage) in place?

Risk Findings

The risk **Missing Vault (Secret Storage)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Vault (Secret Storage) in the threat model (referencing asset **PostgreSQL Database** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.
missing-vault@postgresql-db

Unchecked

Missing Web Application Firewall (WAF): 1 / 1 Risk

Description (Tampering): CWE 1008

To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Impact

If this risk is unmitigated, attackers might be able to apply standard attack pattern tests at great speed without any filtering.

Detection Logic

In-scope web-services and/or web-applications accessed across a network trust boundary not having a Web Application Firewall (WAF) in front of them.

Risk Rating

The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

False Positives

Targets only accessible via WAFs or reverse proxies containing a WAF component (like ModSecurity) can be considered as false positives after individual review.

Mitigation (Operations): Web Application Firewall (WAF)

Consider placing a Web Application Firewall (WAF) in front of the web-services and/or web-applications. For cloud environments many cloud providers offer pre-configured WAFs. Even reverse proxies can be enhances by a WAF component via ModSecurity plugins.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Virtual_Patching_Cheat_Sheet

Check

Is a Web Application Firewall (WAF) in place?

Risk Findings

The risk **Missing Web Application Firewall (WAF)** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Missing Web Application Firewall (WAF) risk at **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-waf@api-server

Unchecked

Mixed Targets on Shared Runtime: 1 / 1 Risk

Description (Elevation of Privilege): CWE 1008

Different attacker targets (like frontend and backend/datastore components) should not be running on the same shared (underlying) runtime.

Impact

If this risk is unmitigated, attackers successfully attacking other components of the system might have an easy path towards more valuable targets, as they are running on the same shared runtime.

Detection Logic

Shared runtime running technical assets of different trust-boundaries is at risk. Also mixing backend/datastore with frontend components on the same shared runtime is considered a risk.

Risk Rating

The risk rating (low or medium) depends on the confidentiality, integrity, and availability rating of the technical asset running on the shared runtime.

False Positives

When all assets running on the shared runtime are hardened and protected to the same extend as if all were containing/processing highly sensitive data.

Mitigation (Operations): Runtime Separation

Use separate runtime environments for running different target components or apply similar separation styles to prevent load- or breach-related problems originating from one more attacker-facing asset impacts also the other more critical rated backend/datastore assets.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Attack_Surface_Analysis_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Mixed Targets on Shared Runtime** was found **1 time** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Mixed Targets on Shared Runtime named **Docker Host** might enable attackers moving from one less valuable target to a more valuable one: Exploitation likelihood is *Unlikely* with *Medium* impact.

mixed-targets-on-shared-runtime@docker-host

Accepted 2026-04-10 Infrastructure Team VAULT-95

Docker containers run without privileged mode and with no host path mounts except the named volumes. The Docker socket is not mounted into any container. Risk is accepted for this demo environment.

Unencrypted Technical Assets: 5 / 5 Risks

Description (Information Disclosure): CWE 311

Due to the confidentiality rating of the technical asset itself and/or the processed data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Impact

If this risk is unmitigated, attackers might be able to access unencrypted data when successfully compromising sensitive components.

Detection Logic

In-scope unencrypted technical assets (excluding reverse-proxy, load-balancer, waf, ids, ips and embedded components like library) storing data assets rated at least as confidential or critical. For technical assets storing data assets rated as strictly-confidential or mission-critical the encryption must be of type data-with-enduser-individual-key.

Risk Rating

Depending on the confidentiality rating of the stored data-assets either medium or high risk.

False Positives

When all sensitive data stored within the asset is already fully encrypted on document or data level.

Mitigation (Operations): Encryption of Technical Asset

Apply encryption to the technical asset.

ASVS Chapter: V6 - Stored Cryptography Verification Requirements
Cheat Sheet: Cryptographic_Storage_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **Unencrypted Technical Assets** was found **5 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unencrypted Technical Asset named **API Server**: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@api-server

Unchecked

Unencrypted Technical Asset named **Browser SPA**: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@browser-spa

Unchecked

Unencrypted Technical Asset named **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@redis-cache

Unchecked

Unencrypted Technical Asset named **PostgreSQL Database** missing enduser-individual encryption with data-with-enduser-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@postgresql-db

Accepted 2026-04-15 Infrastructure Team VAULT-89

The PostgreSQL volume runs on a host-level LUKS-encrypted partition in all production deployments. The Threagile finding is acknowledged as informational for this demo environment which uses unencrypted storage. Production database TDE via pgcrypto will be evaluated in Q3.

Unencrypted Technical Asset named **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@minio-storage

in Progress 2026-04-18 Security Team VAULT-110

MinIO server-side encryption (SSE-S3) is being evaluated. Default minioadmin credentials will be rotated before production deployment. This finding is a CRITICAL blocker for launch. Tracked in VAULT-110.

DoS-risky Access Across Trust-Boundary: 3 / 3 Risks

Description (Denial of Service): CWE 400

Assets accessed across trust boundaries with critical or mission-critical availability rating are more prone to Denial-of-Service (DoS) risks.

Impact

If this risk remains unmitigated, attackers might be able to disturb the availability of important parts of the system.

Detection Logic

In-scope technical assets (excluding load-balancer) with availability rating of critical or higher which have incoming data-flows across a network trust-boundary (excluding devops usage).

Risk Rating

Matching technical assets with availability rating of critical or higher are at low risk. When the availability rating is mission-critical and neither a VPN nor IP filter for the incoming data-flow nor redundancy for the asset is applied, the risk-rating is considered medium.

False Positives

When the accessed target operations are not time- or resource-consuming.

Mitigation (Operations): Anti-DoS Measures

Apply anti-DoS techniques like throttling and/or per-client load blocking with quotas. Also for maintenance access routes consider applying a VPN instead of public reachable interfaces. Generally applying redundancy on the targeted technical asset reduces the risk of DoS.

ASVS Chapter: V1 - Architecture, Design and Threat Modeling Requirements
Cheat Sheet: Denial_of_Service_Cheat_Sheet

Check

Are recommendations from the linked cheat sheet and referenced ASVS chapter applied?

Risk Findings

The risk **DoS-risky Access Across Trust-Boundary** was found **3 times** in the analyzed architecture to be potentially possible. Each spot should be checked individually by reviewing the implementation whether all controls have been applied properly in order to mitigate each risk.
Risk finding paragraphs are clickable and link to the corresponding chapter.

Low Risk Severity

Denial-of-Service risky access of **API Server** by **Browser SPA** via **HTTPS to Nginx** forwarded via **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *Low* impact.
dos-risky-access-across-trust-boundary@api-server@browser-spa@browser-spa>https-to-nginx

Unchecked

Denial-of-Service risky access of **Nginx Reverse Proxy** by **Browser SPA** via **HTTPS to Nginx**: Exploitation likelihood is *Unlikely* with *Low* impact.
dos-risky-access-across-trust-boundary@nginx-proxy@browser-spa@browser-spa>https-to-nginx

Unchecked

Denial-of-Service risky access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Unlikely* with *Low* impact.
dos-risky-access-across-trust-boundary@postgresql-db@api-server@api-server>postgresql-connection

Unchecked

Identified Risks by Technical Asset

In total **34 potential risks** have been identified during the threat modeling process of which **0 are rated as critical**, **1 as high**, **12 as elevated**, **18 as medium**, and **3 as low**.

These risks are distributed across **6 in-scope technical assets**. The following sub-chapters of this section describe each identified risk grouped by technical asset. The RAA value of a technical asset is the calculated "Relative Attacker Attractiveness" value in percent.

API Server: 15 / 15 Risks

Description

Node.js/Express REST API. Handles authentication (JWT/bcrypt), CRUD on notes, file uploads to MinIO. Bridges the frontend and backend networks. Single most security-critical component — compromise equals full data access.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

High Risk Severity

SQL/NoSQL-Injection risk at **API Server** against database **PostgreSQL Database** via **PostgreSQL Connection**: Exploitation likelihood is *Very Likely* with *High* impact.

sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Unchecked

Elevated Risk Severity

Missing Authentication covering communication link **HTTP to API Server** from **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Likely* with *High* impact.

missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Unencrypted Communication named **MinIO File Storage** between **API Server** and **MinIO Object Storage** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unencrypted Communication named **PostgreSQL Connection** between **API Server** and **PostgreSQL Database** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Unencrypted Communication named **Redis Session Store** between **API Server** and **Redis Cache** transferring authentication data (like credentials, token, session-id, etc.): Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@api-server>redis-session-store@api-server@redis-cache

Unchecked

Path-Traversal risk at **API Server** against filesystem **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Very Likely* with *Medium* impact.

path-traversal@api-server@minio-storage@api-server>minio-file-storage

Unchecked

SQL/NoSQL-Injection risk at **API Server** against database **Redis Cache** via **Redis Session Store**: Exploitation likelihood is *Very Likely* with *Medium* impact.

sql-nosql-injection@api-server@redis-cache@api-server>redis-session-store

Unchecked

Missing Hardening risk at **API Server**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@api-server

Unchecked

Server-Side Request Forgery (SSRF) risk at **API Server** server-side web-requesting the target **MinIO Object Storage** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **API Server**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@api-server

Unchecked

Unencrypted Technical Asset named **API Server**: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@api-server

Unchecked

Missing Build Infrastructure in the threat model (referencing asset **API Server** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-build-infrastructure@api-server

Unchecked

Missing Two-Factor Authentication covering communication link **HTTP to API Server** from **Browser SPA** forwarded via **Nginx Reverse Proxy** to **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Unchecked

Missing Web Application Firewall (WAF) risk at **API Server**: Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-waf@api-server

Unchecked

Low Risk Severity

Denial-of-Service risky access of **API Server** by **Browser SPA** via **HTTPS to Nginx** forwarded via **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *Low* impact.

dos-risky-access-across-trust-boundary@api-server@browser-spa@browser-spa>https-to-nginx

Unchecked

Asset Information

ID:	api-server
Type:	process
Usage:	business
RAA:	77 %
Size:	service
Technology:	web-service-rest
Tags:	express, jwt, nodejs
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	false
Data Processed:	File Attachments, Note Content, Session Tokens, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	The API processes all user data. It bridges both Docker networks — a compromised API container can pivot into the internal data tier.	

Outgoing Communication Links: 3
Target technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (outgoing)
Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Target:	Redis Cache
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Session Tokens
Data Received:	Session Tokens

PostgreSQL Connection (outgoing)
Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Target:	PostgreSQL Database
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	Note Content, User Credentials
Data Received:	Note Content, User Credentials

MinIO File Storage (outgoing)
INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net

can read all stored files.

Target:	MinIO Object Storage
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments
Data Received:	File Attachments

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (incoming)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Source:	Nginx Reverse Proxy
Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

Nginx Reverse Proxy: 5 / 5 Risks

Description

Nginx terminates TLS and forwards all API traffic to the Express server over plain HTTP (port 3000). This is the DMZ entry point.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Server-Side Request Forgery (SSRF) risk at **Nginx Reverse Proxy** server-side web-requesting the target **API Server** via **HTTP to API Server**: Exploitation likelihood is *Likely* with *Medium* impact.

server-side-request-forgery@nginx-proxy@api-server@nginx-proxy>http-to-api-server

Unchecked

Unencrypted Communication named **HTTP to API Server** between **Nginx Reverse Proxy** and **API Server**: Exploitation likelihood is *Likely* with *High* impact.

unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

in Progress 2026-04-18 Security Team VAULT-102

Migrating Nginx-to-API communication to mTLS. A separate CA will be provisioned inside the Docker network. PR #47 is in review; target completion sprint 22.

Medium Risk Severity

Container Base Image Backdooring risk at **Nginx Reverse Proxy**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@nginx-proxy

Unchecked

Missing Identity Store in the threat model (referencing asset **Nginx Reverse Proxy** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-identity-store@nginx-proxy

Unchecked

Low Risk Severity

Denial-of-Service risky access of **Nginx Reverse Proxy** by **Browser SPA** via **HTTPS to Nginx**: Exploitation likelihood is *Unlikely* with *Low* impact.

dos-risky-access-across-trust-boundary@nginx-proxy@browser-spa@browser-spa>https-to-nginx

Unchecked

Asset Information

ID:	nginx-proxy
Type:	process
Usage:	business
RAA:	2 %
Size:	service
Technology:	reverse-proxy
Tags:	nginx, tls-termination
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	internal	(rated 2 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Nginx is the single external entry point. Its compromise exposes all downstream services. Availability is critical — outage = full service outage.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTP to API Server (outgoing)

INTENTIONAL MISCONFIGURATION: Traffic between Nginx and the API server uses plaintext HTTP. An attacker with access to the Docker network (container escape, compromised sidecar) can intercept credentials, session tokens, and note content in transit.

Target:	API Server

Protocol:	http
Encrypted:	false
Authentication:	none
Authorization:	none
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (incoming)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Source:	Browser SPA
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	enduser-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments, Note Content, User Credentials
Data Sent:	File Attachments, Note Content

PostgreSQL Database: 6 / 6 Risks

Description

Primary relational datastore. Contains users table (email + bcrypt password hash), notes table (title + content), and attachments metadata. INTENTIONAL MISCONFIGURATION: No encryption at rest (encryption: none).

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Elevated Risk Severity

Missing Hardening risk at **PostgreSQL Database**: Exploitation likelihood is *Likely* with *Medium* impact.

missing-hardening@postgresql-db

Unchecked

Unguarded Direct Datastore Access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Likely* with *Medium* impact.

unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Unchecked

Medium Risk Severity

Container Base Image Backdooring risk at **PostgreSQL Database**: Exploitation likelihood is *Unlikely* with *High* impact.

container-baseimage-backdooring@postgresql-db

Unchecked

Missing Vault (Secret Storage) in the threat model (referencing asset **PostgreSQL Database** as an example): Exploitation likelihood is *Unlikely* with *Medium* impact.

missing-vault@postgresql-db

Unchecked

Unencrypted Technical Asset named **PostgreSQL Database** missing enduser-individual encryption with data-with-enduser-individual-key: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@postgresql-db

Accepted 2026-04-15 Infrastructure Team VAULT-89

The PostgreSQL volume runs on a host-level LUKS-encrypted partition in all production deployments. The Threagile finding is acknowledged as informational for this demo environment which uses unencrypted storage. Production database TDE via pgcrypto will be evaluated in Q3.

Low Risk Severity

Denial-of-Service risky access of **PostgreSQL Database** by **API Server** via **PostgreSQL Connection**: Exploitation likelihood is *Unlikely* with *Low* impact.

dos-risky-access-across-trust-boundary@postgresql-db@api-server@api-server>postgresql-connection

Unchecked

Asset Information

ID:	postgresql-db
Type:	datastore
Usage:	business
RAA:	100 %
Size:	service
Technology:	database
Tags:	postgresql, relational
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	Note Content, User Credentials
Data Stored:	Note Content, User Credentials
Formats Accepted:	JSON

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	critical	(rated 4 in scale of 5)
CIA-Justification:	Holds all user PII (email, hashed passwords) and all note content. Strictly confidential — unencrypted disk access exposes all data.	

Incoming Communication Links: 1
Source technical asset names are clickable and link to the corresponding chapter.

PostgreSQL Connection (incoming)
Application-level SQL queries for user auth, notes CRUD. Uses parameterized queries (pg library) to prevent SQL injection. No TLS on the connection — traffic is plaintext within backend-net.

Source:	API Server
Protocol:	sql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Note Content, User Credentials
Data Sent:	Note Content, User Credentials

Browser SPA: 1 / 1 Risk

Description

React Single Page Application served from Nginx. Users authenticate, manage notes, and upload files through this interface.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Unencrypted Technical Asset named **Browser SPA**: Exploitation likelihood is *Unlikely* with *High* impact.

unencrypted-asset@browser-spa

Unchecked

Asset Information

ID:	browser-spa
Type:	external-entity
Usage:	business
RAA:	46 %
Size:	application
Technology:	browser
Tags:	react, spa
Internet:	true
Machine:	physical
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	true
Client by Human:	true
Data Processed:	File Attachments, Note Content, User Credentials
Data Stored:	none
Formats Accepted:	JSON

Asset Rating

Owner:	End Users

Confidentiality:	public	(rated 1 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Client-side code is public; the trust boundary here is user data entered into the browser. XSS would allow an attacker to exfiltrate session tokens.	

Outgoing Communication Links: 1

Target technical asset names are clickable and link to the corresponding chapter.

HTTPS to Nginx (outgoing)

User-initiated HTTPS requests carrying credentials (login) and note content (create/edit). This is the only encrypted link in the chain.

Target:	Nginx Reverse Proxy
Protocol:	https
Encrypted:	true
Authentication:	credentials
Authorization:	enduser-identity-propagation
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Sent:	File Attachments, Note Content, User Credentials
Data Received:	File Attachments, Note Content

MinIO Object Storage: 3 / 3 Risks

Description

S3-compatible object store for file attachments. INTENTIONAL MISCONFIGURATION: Uses default minioadmin/minioadmin credentials (visible in docker-compose.yml) and stores files without encryption at rest. HTTP-only (no TLS) within the Docker network.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@minio-storage

Unchecked

Unguarded Direct Datastore Access of **MinIO Object Storage** by **API Server** via **MinIO File Storage**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>minio-file-storage@api-server@minio-storage

Unchecked

Unencrypted Technical Asset named **MinIO Object Storage**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@minio-storage

in Progress 2026-04-18 Security Team VAULT-110

MinIO server-side encryption (SSE-S3) is being evaluated. Default minioadmin credentials will be rotated before production deployment. This finding is a CRITICAL blocker for launch. Tracked in VAULT-110.

Asset Information

ID:	minio-storage
Type:	datastore
Usage:	business
RAA:	33 %
Size:	service
Technology:	file-server
Tags:	minio, object-storage, s3
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false

Redundant:	false
Custom-Developed:	false
Client by Human:	false
Data Processed:	File Attachments
Data Stored:	File Attachments
Formats Accepted:	File

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	important	(rated 3 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Stores user file uploads which may contain sensitive documents. Default credentials + no encryption = trivial exfiltration by any process on backend-net.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

MinIO File Storage (incoming)

INTENTIONAL MISCONFIGURATION: S3 API calls to MinIO over plain HTTP. Default minioadmin/minioadmin credentials are hardcoded in compose. Any container on backend-net can read all stored files.

Source:	API Server
Protocol:	http
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	intentional-misconfiguration
VPN:	false
IP-Filtered:	false
Data Received:	File Attachments
Data Sent:	File Attachments

Redis Cache: 3 / 3 Risks

Description

In-memory store used as session store and JWT revocation list. INTENTIONAL MISCONFIGURATION: No encryption at rest. If Redis persistence is enabled and the RDB file is exfiltrated, session tokens are exposed.

Identified Risks of Asset

Risk finding paragraphs are clickable and link to the corresponding chapter.

Medium Risk Severity

Container Base Image Backdooring risk at **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

container-baseimage-backdooring@redis-cache

Unchecked

Unencrypted Technical Asset named **Redis Cache**: Exploitation likelihood is *Unlikely* with *Medium* impact.

unencrypted-asset@redis-cache

Unchecked

Unguarded Direct Datastore Access of **Redis Cache** by **API Server** via **Redis Session Store**: Exploitation likelihood is *Likely* with *Low* impact.

unguarded-direct-datastore-access@api-server>redis-session-store@api-server@redis-cache

Unchecked

Asset Information

ID:	redis-cache
Type:	datastore
Usage:	business
RAA:	39 %
Size:	component
Technology:	database
Tags:	cache, redis, session-store
Internet:	false
Machine:	container
Encryption:	none
Multi-Tenant:	false
Redundant:	false
Custom-Developed:	false

Client by Human:	false
Data Processed:	Session Tokens
Data Stored:	Session Tokens
Formats Accepted:	none of the special data formats accepted

Asset Rating

Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	operational	(rated 2 in scale of 5)
Availability:	important	(rated 3 in scale of 5)
CIA-Justification:	Session token exposure enables account takeover. Non-persistent by default but the container volume could persist RDB snapshots.	

Incoming Communication Links: 1

Source technical asset names are clickable and link to the corresponding chapter.

Redis Session Store (incoming)

Session token storage and JWT revocation list. Password-protected (requirepass) but no TLS.

Source:	API Server
Protocol:	nosql-access-protocol
Encrypted:	false
Authentication:	credentials
Authorization:	technical-user
Read-Only:	false
Usage:	business
Tags:	none
VPN:	false
IP-Filtered:	false
Data Received:	Session Tokens
Data Sent:	Session Tokens

Identified Data Breach Probabilities by Data Asset

In total **34 potential risks** have been identified during the threat modeling process of which **0 are rated as critical**, **1 as high**, **12 as elevated**, **18 as medium**, and **3 as low**.

These risks are distributed across **4 data assets**. The following sub-chapters of this section describe the derived data breach probabilities grouped by data asset.

Technical asset names and risk IDs are clickable and link to the corresponding chapter.

File Attachments: 17 / 17 Risks

Binary file uploads attached to notes. May include documents, images, or other sensitive files depending on note usage.

ID:	file-attachments	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	important	(rated 3 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Files may contain confidential content. MinIO's default credentials (minioadmin) make all stored files accessible to anyone on the network.	
Processed by:	API Server, Browser SPA, MinIO Object Storage, Nginx Reverse Proxy	
Stored by:	MinIO Object Storage	
Sent via:	MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Received via:	MinIO File Storage, HTTPS to Nginx, HTTP to API Server	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 17 remaining risks:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@minio-storage

Probable: container-baseimage-backdooring@nginx-proxy

Probable: path-traversal@api-server@minio-storage@api-server>minio-file-storage

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: server-side-request-forgery@nginx-proxy@api-server@nginx-proxy>http-to-api-server

Possible: unencrypted-communication@api-server>minio-file-storage@api-server@minio-storage

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-hardening@api-server

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@api-server

Improbable: unencrypted-asset@browser-spa

Improbable: unguarded-direct-datastore-access@api-server>minio-file-storage@api-server@minio-storage

Improbable: mixed-targets-on-shared-runtime@docker-host

Improbable: unencrypted-asset@minio-storage

Note Content: 18 / 18 Risks

User-authored note text — may include personal, financial, or health information depending on how users employ the application.

ID:	note-content	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Notes may contain highly sensitive personal information. Integrity is critical because corrupted notes lose user trust.	
Processed by:	API Server, Browser SPA, Nginx Reverse Proxy, PostgreSQL Database	
Stored by:	PostgreSQL Database	
Sent via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Received via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 18 remaining risks:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@nginx-proxy

Probable: container-baseimage-backdooring@postgresql-db

Probable: sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: server-side-request-forgery@nginx-proxy@api-server@nginx-proxy>http-to-api-server

Possible: unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-hardening@api-server

Improbable: missing-hardening@postgresql-db

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@api-server

Improbable: unencrypted-asset@browser-spa

Improbable: unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Improbable: mixed-targets-on-shared-runtime@docker-host

Improbable: unencrypted-asset@postgresql-db

Session Tokens: 14 / 14 Risks

JWT tokens and Redis session keys used to maintain authenticated state. Theft allows session hijacking without knowing the user's password.

ID:	session-tokens	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	internal	
Owner:	VaultNote Team	
Confidentiality:	confidential	(rated 4 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Session token compromise enables full session hijacking. Short TTLs and revocation lists mitigate but do not eliminate the risk.	
Processed by:	API Server, Redis Cache	
Stored by:	Redis Cache	
Sent via:	Redis Session Store	
Received via:	Redis Session Store	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 14 remaining risks:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@redis-cache

Probable: sql-nosql-injection@api-server@redis-cache@api-server>redis-session-store

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: unencrypted-communication@api-server>redis-session-store@api-server@redis-cache

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-hardening@api-server

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@api-server

Improbable: unencrypted-asset@redis-cache

Improbable: unguarded-direct-datastore-access@api-server>redis-session-store@api-server@redis-cache

Improbable: mixed-targets-on-shared-runtime@docker-host

User Credentials: 18 / 18 Risks

Email addresses and bcrypt-hashed passwords used for authentication. Compromise allows full account takeover.

ID:	user-credentials	
Usage:	business	
Quantity:	many	
Tags:	none	
Origin:	customer	
Owner:	VaultNote Team	
Confidentiality:	strictly-confidential	(rated 5 in scale of 5)
Integrity:	critical	(rated 4 in scale of 5)
Availability:	operational	(rated 2 in scale of 5)
CIA-Justification:	Credentials grant full account access. A breach leaks all users' identities and enables account takeover across every note and attachment.	
Processed by:	API Server, Browser SPA, Nginx Reverse Proxy, PostgreSQL Database	
Stored by:	PostgreSQL Database	
Sent via:	PostgreSQL Connection, HTTPS to Nginx, HTTP to API Server	
Received via:	PostgreSQL Connection	
Data Breach:	**probable**	
Data Breach Risks:	This data asset has data breach potential because of 18 remaining risks:	

Probable: container-baseimage-backdooring@api-server

Probable: container-baseimage-backdooring@nginx-proxy

Probable: container-baseimage-backdooring@postgresql-db

Probable: sql-nosql-injection@api-server@postgresql-db@api-server>postgresql-connection

Possible: missing-authentication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: missing-authentication-second-factor@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Possible: server-side-request-forgery@api-server@minio-storage@api-server>minio-file-storage

Possible: server-side-request-forgery@nginx-proxy@api-server@nginx-proxy>http-to-api-server

Possible: unencrypted-communication@api-server>postgresql-connection@api-server@postgresql-db

Possible: unencrypted-communication@nginx-proxy>http-to-api-server@nginx-proxy@api-server

Improbable: missing-hardening@api-server

Improbable: missing-hardening@postgresql-db

Improbable: missing-waf@api-server

Improbable: unencrypted-asset@api-server

Improbable: unencrypted-asset@browser-spa

Improbable: unguarded-direct-datastore-access@api-server>postgresql-connection@api-server@postgresql-db

Improbable: mixed-targets-on-shared-runtime@docker-host

Improbable: unencrypted-asset@postgresql-db

Trust Boundaries

In total **4 trust boundaries** have been modeled during the threat modeling process.

Application Network

Internal Docker bridge network (frontend-net). Contains the API server which is reachable from Nginx but also bridges into backend-net.

ID:	application-network
Type:	network-on-prem
Tags:	none
Assets inside:	API Server
Boundaries nested:	none

DMZ

Demilitarized zone containing the Nginx reverse proxy. Accessible from the internet but isolated from the application and data tiers.

ID:	dmz
Type:	network-on-prem
Tags:	none
Assets inside:	Nginx Reverse Proxy
Boundaries nested:	none

Data Tier

Internal Docker bridge network (backend-net, marked internal: true). No external routing. Contains all datastores. The API server has a NIC on this network and acts as the sole legitimate access path.

ID:	data-tier
Type:	network-on-prem
Tags:	none
Assets inside:	MinIO Object Storage, PostgreSQL Database, Redis Cache
Boundaries nested:	none

Internet

Untrusted external network. All traffic from this zone must cross the Nginx TLS termination point before entering the DMZ.

ID:	internet
Type:	network-on-prem
Tags:	none
Assets inside:	Browser SPA
Boundaries nested:	none

Shared Runtimes

In total **1 shared runtime** has been modeled during the threat modeling process.

Docker Host
All containers run on the same Docker host. A container escape or privileged container could access the host filesystem including Docker socket and database volumes.

ID:	docker-host
Tags:	container-runtime, docker
Assets running:	Nginx Reverse Proxy, API Server, PostgreSQL Database, Redis Cache, MinIO Object Storage

Risk Rules Checked by Threagile

Threagile Version: 1.0.0
Threagile Build Timestamp: 20240730113903
Threagile Execution Timestamp: 20260418084715
Model Filename: /app/work/threagile.yaml
Model Hash (SHA256): de51452b7150a97f2a7791a460340a18944b8da7c88d9859d7a10b4797ca74df

Threagile (see https://threagile.io for more details) is an open-source toolkit for agile threat modeling, created by Christian Schneider (https://christian-schneider.net): It allows to model an architecture with its assets in an agile fashion as a YAML file directly inside the IDE. Upon execution of the Threagile toolkit all standard risk rules (as well as individual custom rules if present) are checked against the architecture model. At the time the Threagile toolkit was executed on the model input file the following risk rules were checked:

Accidental Secret Leak
accidental-secret-leak

STRIDE:	Information Disclosure
Description:	Sourcecode repositories (including their histories) as well as artifact registries can accidentally contain secrets like checked-in or packaged-in passwords, API tokens, certificates, crypto keys, etc.
Detection:	In-scope sourcecode repositories and artifact registries.
Rating:	The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Code Backdooring
code-backdooring

STRIDE:	Tampering
Description:	For each build-pipeline component Code Backdooring risks might arise where attackers compromise the build-pipeline in order to let backdoored artifacts be shipped into production. Aside from direct code backdooring this includes backdooring of dependencies and even of more lower-level build infrastructure, like backdooring compilers (similar to what the XcodeGhost malware did) or dependencies.
Detection:	In-scope development relevant technical assets which are either accessed by out-of-scope unmanaged developer clients and/or are directly accessed by any kind of internet-located (non-VPN) component or are themselves directly located on the internet.
Rating:	The risk rating depends on the confidentiality and integrity rating of the code being handled and deployed as well as the placement/calling of this technical asset on/from the internet.

Container Base Image Backdooring
container-baseimage-backdooring

STRIDE: Tampering
Description: When a technical asset is built using container technologies, Base Image Backdooring risks might arise where base images and other layers used contain vulnerable components or backdoors.
Detection: In-scope technical assets running as containers.
Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets.

Container Platform Escape
container-platform-escape

STRIDE: Elevation of Privilege
Description: Container platforms are especially interesting targets for attackers as they host big parts of a containerized runtime infrastructure. When not configured and operated with security best practices in mind, attackers might exploit a vulnerability inside an container and escape towards the platform as highly privileged users. These scenarios might give attackers capabilities to attack every other container as owning the container platform (via container escape attacks) equals to owning every container.
Detection: In-scope container platforms.
Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Cross-Site Request Forgery (CSRF)
cross-site-request-forgery

STRIDE: Spoofing
Description: When a web application is accessed via web protocols Cross-Site Request Forgery (CSRF) risks might arise.
Detection: In-scope web applications accessed via typical web access protocols.
Rating: The risk rating depends on the integrity rating of the data sent across the communication link.

Cross-Site Scripting (XSS)
cross-site-scripting

STRIDE: Tampering
Description: For each web application Cross-Site Scripting (XSS) risks might arise. In terms of the overall risk level take other applications running on the same domain into account as well.
Detection: In-scope web applications.
Rating: The risk rating depends on the sensitivity of the data processed or stored in the web application.

DoS-risky Access Across Trust-Boundary

dos-risky-access-across-trust-boundary

STRIDE: Denial of Service

Description: Assets accessed across trust boundaries with critical or mission-critical availability rating are more prone to Denial-of-Service (DoS) risks.

Detection: In-scope technical assets (excluding load-balancer) with availability rating of critical or higher which have incoming data-flows across a network trust-boundary (excluding devops usage).

Rating: Matching technical assets with availability rating of critical or higher are at low risk. When the availability rating is mission-critical and neither a VPN nor IP filter for the incoming data-flow nor redundancy for the asset is applied, the risk-rating is considered medium.

Incomplete Model

incomplete-model

STRIDE: Information Disclosure

Description: When the threat model contains unknown technologies or transfers data over unknown protocols, this is an indicator for an incomplete model.

Detection: All technical assets and communication links with technology type or protocol type specified as unknown.

Rating: low

LDAP-Injection

ldap-injection

STRIDE: Tampering

Description: When an LDAP server is accessed LDAP-Injection risks might arise. The risk rating depends on the sensitivity of the LDAP server itself and of the data assets processed or stored.

Detection: In-scope clients accessing LDAP servers via typical LDAP access protocols.

Rating: The risk rating depends on the sensitivity of the LDAP server itself and of the data assets processed or stored.

Missing Authentication

missing-authentication

STRIDE: Elevation of Privilege

Description: Technical assets (especially multi-tenant systems) should authenticate incoming requests when the asset processes or stores sensitive data.

Detection: In-scope technical assets (except load-balancer, reverse-proxy, service-registry, waf, ids, and ips and in-process calls) should authenticate incoming requests when the asset processes or stores sensitive data. This is especially the case for all multi-tenant assets (there even non-sensitive ones).

Rating: The risk rating (medium or high) depends on the sensitivity of the data sent across

the communication link. Monitoring callers are exempted from this risk.

Missing Two-Factor Authentication (2FA)

missing-authentication-second-factor

STRIDE: Elevation of Privilege

Description: Technical assets (especially multi-tenant systems) should authenticate incoming requests with two-factor (2FA) authentication when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by humans.

Detection: In-scope technical assets (except load-balancer, reverse-proxy, waf, ids, and ips) should authenticate incoming requests via two-factor authentication (2FA) when the asset processes or stores highly sensitive data (in terms of confidentiality, integrity, and availability) and is accessed by a client used by a human user.

Rating: medium

Missing Build Infrastructure

missing-build-infrastructure

STRIDE: Tampering

Description: The modeled architecture does not contain a build infrastructure (devops-client, sourcecode-repo, build-pipeline, etc.), which might be the risk of a model missing critical assets (and thus not seeing their risks). If the architecture contains custom-developed parts, the pipeline where code gets developed and built needs to be part of the model.

Detection: Models with in-scope custom-developed parts missing in-scope development (code creation) and build infrastructure components (devops-client, sourcecode-repo, build-pipeline, etc.).

Rating: The risk rating depends on the highest sensitivity of the in-scope assets running custom-developed parts.

Missing Cloud Hardening

missing-cloud-hardening

STRIDE: Tampering

Description: Cloud components should be hardened according to the cloud vendor best practices. This affects their configuration, auditing, and further areas.

Detection: In-scope cloud components (either residing in cloud trust boundaries or more specifically tagged with cloud provider types).

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Missing File Validation

missing-file-validation

STRIDE: Spoofing

Description: When a technical asset accepts files, these input files should be strictly validated about filename and type.
Detection: In-scope technical assets with custom-developed code accepting file data formats.
Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Missing Hardening
missing-hardening

STRIDE: Tampering
Description: Technical assets with a Relative Attacker Attractiveness (RAA) value of 55 % or higher should be explicitly hardened taking best practices and vendor hardening guides into account.
Detection: In-scope technical assets with RAA values of 55 % or higher. Generally for high-value targets like datastores, application servers, identity providers and ERP systems this limit is reduced to 40 %
Rating: The risk rating depends on the sensitivity of the data processed or stored in the technical asset.

Missing Identity Propagation
missing-identity-propagation

STRIDE: Elevation of Privilege
Description: Technical assets (especially multi-tenant systems), which usually process data for endusers should authorize every request based on the identity of the enduser when the data flow is authenticated (i.e. non-public). For DevOps usages at least a technical-user authorization is required.
Detection: In-scope service-like technical assets which usually process data based on enduser requests, if authenticated (i.e. non-public), should authorize incoming requests based on the propagated enduser identity when their rating is sensitive. This is especially the case for all multi-tenant assets (there even less-sensitive rated ones). DevOps usages are exempted from this risk.
Rating: The risk rating (medium or high) depends on the confidentiality, integrity, and availability rating of the technical asset.

Missing Identity Provider Isolation
missing-identity-provider-isolation

STRIDE: Elevation of Privilege
Description: Highly sensitive identity provider assets and their identity datastores should be isolated from other assets by their own network segmentation trust-boundary (execution-environment boundaries do not count as network isolation).
Detection: In-scope identity provider assets and their identity datastores when surrounded by other (not identity-related) assets (without a network trust-boundary in-between).

This risk is especially prevalent when other non-identity related assets are within the same execution environment (i.e. same database or same application server).

Rating: Default is high impact. The impact is increased to very-high when the asset missing the trust-boundary protection is rated as strictly-confidential or mission-critical.

Missing Identity Store

missing-identity-store

STRIDE: Spoofing

Description: The modeled architecture does not contain an identity store, which might be the risk of a model missing critical assets (and thus not seeing their risks).

Detection: Models with authenticated data-flows authorized via enduser-identity missing an in-scope identity store.

Rating: The risk rating depends on the sensitivity of the enduser-identity authorized technical assets and their data assets processed and stored.

Missing Network Segmentation

missing-network-segmentation

STRIDE: Elevation of Privilege

Description: Highly sensitive assets and/or datastores residing in the same network segment than other lower sensitive assets (like webservers or content management systems etc.) should be better protected by a network segmentation trust-boundary.

Detection: In-scope technical assets with high sensitivity and RAA values as well as datastores when surrounded by assets (without a network trust-boundary in-between) which are of type client-system, web-server, web-application, cms, web-service-rest, web-service-soap, build-pipeline, sourcecode-repository, monitoring, or similar and there is no direct connection between these (hence no requirement to be so close to each other).

Rating: Default is low risk. The risk is increased to medium when the asset missing the trust-boundary protection is rated as strictly-confidential or mission-critical.

Missing Vault (Secret Storage)

missing-vault

STRIDE: Information Disclosure

Description: In order to avoid the risk of secret leakage via config files (when attacked through vulnerabilities being able to read files like Path-Traversal and others), it is best practice to use a separate hardened process with proper authentication, authorization, and audit logging to access config secrets (like credentials, private keys, client certificates, etc.). This component is usually some kind of Vault.

Detection: Models without a Vault (Secret Storage).

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Missing Vault Isolation
missing-vault-isolation

STRIDE: Elevation of Privilege

Description: Highly sensitive vault assets and their datastores should be isolated from other assets by their own network segmentation trust-boundary (execution-environment boundaries do not count as network isolation).

Detection: In-scope vault assets when surrounded by other (not vault-related) assets (without a network trust-boundary in-between). This risk is especially prevalent when other non-vault related assets are within the same execution environment (i.e. same database or same application server).

Rating: Default is medium impact. The impact is increased to high when the asset missing the trust-boundary protection is rated as strictly-confidential or mission-critical.

Missing Web Application Firewall (WAF)
missing-waf

STRIDE: Tampering

Description: To have a first line of filtering defense, security architectures with web-services or web-applications should include a WAF in front of them. Even though a WAF is not a replacement for security (all components must be secure even without a WAF) it adds another layer of defense to the overall system by delaying some attacks and having easier attack alerting through it.

Detection: In-scope web-services and/or web-applications accessed across a network trust boundary not having a Web Application Firewall (WAF) in front of them.

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Mixed Targets on Shared Runtime
mixed-targets-on-shared-runtime

STRIDE: Elevation of Privilege

Description: Different attacker targets (like frontend and backend/datastore components) should not be running on the same shared (underlying) runtime.

Detection: Shared runtime running technical assets of different trust-boundaries is at risk. Also mixing backend/datastore with frontend components on the same shared runtime is considered a risk.

Rating: The risk rating (low or medium) depends on the confidentiality, integrity, and availability rating of the technical asset running on the shared runtime.

Path-Traversal
path-traversal

STRIDE: Information Disclosure

Description: When a filesystem is accessed Path-Traversal or Local-File-Inclusion (LFI) risks might arise. The risk rating depends on the sensitivity of the technical asset itself

and of the data assets processed or stored.

Detection: Filesystems accessed by in-scope callers.

Rating: The risk rating depends on the sensitivity of the data stored inside the technical asset.

Push instead of Pull Deployment

push-instead-of-pull-deployment

STRIDE: Tampering

Description: When comparing push-based vs. pull-based deployments from a security perspective, pull-based deployments improve the overall security of the deployment targets. Every exposed interface of a production system to accept a deployment increases the attack surface of the production system, thus a pull-based approach exposes less attack surface relevant interfaces.

Detection: Models with build pipeline components accessing in-scope targets of deployment (in a non-readonly way) which are not build-related components themselves.

Rating: The risk rating depends on the highest sensitivity of the deployment targets running custom-developed parts.

Search-Query Injection

search-query-injection

STRIDE: Tampering

Description: When a search engine server is accessed Search-Query Injection risks might arise.

Detection: In-scope clients accessing search engine servers via typical search access protocols.

Rating: The risk rating depends on the sensitivity of the search engine server itself and of the data assets processed or stored.

Server-Side Request Forgery (SSRF)

server-side-request-forgery

STRIDE: Information Disclosure

Description: When a server system (i.e. not a client) is accessing other server systems via typical web protocols Server-Side Request Forgery (SSRF) or Local-File-Inclusion (LFI) or Remote-File-Inclusion (RFI) risks might arise.

Detection: In-scope non-client systems accessing (using outgoing communication links) targets with either HTTP or HTTPS protocol.

Rating: The risk rating (low or medium) depends on the sensitivity of the data assets receivable via web protocols from targets within the same network trust-boundary as well on the sensitivity of the data assets receivable via web protocols from the target asset itself. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF.

Service Registry Poisoning

service-registry-poisoning

STRIDE: Spoofing

Description: When a service registry used for discovery of trusted service endpoints Service Registry Poisoning risks might arise.

Detection: In-scope service registries.

Rating: The risk rating depends on the sensitivity of the technical assets accessing the service registry as well as the data assets processed or stored.

SQL/NoSQL-Injection

sql-nosql-injection

STRIDE: Tampering

Description: When a database is accessed via database access protocols SQL/NoSQL-Injection risks might arise. The risk rating depends on the sensitivity technical asset itself and of the data assets processed or stored.

Detection: Database accessed via typical database access protocols by in-scope clients.

Rating: The risk rating depends on the sensitivity of the data stored inside the database.

Unchecked Deployment

unchecked-deployment

STRIDE: Tampering

Description: For each build-pipeline component Unchecked Deployment risks might arise when the build-pipeline does not include established DevSecOps best-practices. DevSecOps best-practices scan as part of CI/CD pipelines for vulnerabilities in source- or byte-code, dependencies, container layers, and dynamically against running test systems. There are several open-source and commercial tools existing in the categories DAST, SAST, and IAST.

Detection: All development-relevant technical assets.

Rating: The risk rating depends on the highest rating of the technical assets and data assets processed by deployment-receiving targets.

Unencrypted Technical Assets

unencrypted-asset

STRIDE: Information Disclosure

Description: Due to the confidentiality rating of the technical asset itself and/or the processed data assets this technical asset must be encrypted. The risk rating depends on the sensitivity technical asset itself and of the data assets stored.

Detection: In-scope unencrypted technical assets (excluding reverse-proxy, load-balancer, waf, ids, ips and embedded components like library) storing data assets rated at least as confidential or critical. For technical assets storing data assets rated as strictly-confidential or mission-critical the encryption must be of type data-with-enduser-individual-key.

Rating: Depending on the confidentiality rating of the stored data-assets either medium or high risk.

Unencrypted Communication
unencrypted-communication

STRIDE: Information Disclosure

Description: Due to the confidentiality and/or integrity rating of the data assets transferred over the communication link this connection must be encrypted.

Detection: Unencrypted technical communication links of in-scope technical assets (excluding monitoring traffic as well as local-file-access and in-process-library-call) transferring sensitive data.

Rating: Depending on the confidentiality rating of the transferred data-assets either medium or high risk.

Unguarded Access From Internet
unguarded-access-from-internet

STRIDE: Elevation of Privilege

Description: Internet-exposed assets must be guarded by a protecting service, application, or reverse-proxy.

Detection: In-scope technical assets (excluding load-balancer) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) when accessed directly from the internet. All web-server, web-application, reverse-proxy, waf, and gateway assets are exempted from this risk when they do not consist of custom developed code and the data-flow only consists of HTTP or FTP protocols. Access from monitoring systems as well as VPN-protected connections are exempted.

Rating: The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

Unguarded Direct Datastore Access
unguarded-direct-datastore-access

STRIDE: Elevation of Privilege

Description: Datastores accessed across trust boundaries must be guarded by some protecting service or application.

Detection: In-scope technical assets of type datastore (except identity-store-ldap when accessed from identity-provider and file-server when accessed via file transfer protocols) with confidentiality rating of confidential (or higher) or with integrity rating of critical (or higher) which have incoming data-flows from assets outside across a network trust-boundary. DevOps config and deployment access is excluded from this risk.

Rating: The matching technical assets are at low risk. When either the confidentiality rating is strictly-confidential or the integrity rating is mission-critical, the risk-rating is considered medium. For assets with RAA values higher than 40 % the risk-rating increases.

Unnecessary Communication Link
unnecessary-communication-link

STRIDE: Elevation of Privilege

Description: When a technical communication link does not send or receive any data assets, this is an indicator for an unnecessary communication link (or for an incomplete model).

Detection: In-scope technical assets' technical communication links not sending or receiving any data assets.

Rating: low

Unnecessary Data Asset
unnecessary-data-asset

STRIDE: Elevation of Privilege

Description: When a data asset is not processed or stored by any data assets and also not transferred by any communication links, this is an indicator for an unnecessary data asset (or for an incomplete model).

Detection: Modelled data assets not processed or stored by any data assets and also not transferred by any communication links.

Rating: low

Unnecessary Data Transfer
unnecessary-data-transfer

STRIDE: Elevation of Privilege

Description: When a technical asset sends or receives data assets, which it neither processes or stores this is an indicator for unnecessarily transferred data (or for an incomplete model). When the unnecessarily transferred data assets are sensitive, this poses an unnecessary risk of an increased attack surface.

Detection: In-scope technical assets sending or receiving sensitive data assets which are neither processed nor stored by the technical asset are flagged with this risk. The risk rating (low or medium) depends on the confidentiality, integrity, and availability rating of the technical asset. Monitoring data is exempted from this risk.

Rating: The risk assessment is depending on the confidentiality and integrity rating of the transferred data asset either low or medium.

Unnecessary Technical Asset
unnecessary-technical-asset

STRIDE: Elevation of Privilege

Description: When a technical asset does not process or store any data assets, this is an

indicator for an unnecessary technical asset (or for an incomplete model). This is also the case if the asset has no communication links (either outgoing or incoming).

Detection: Technical assets not processing or storing any data assets.

Rating: low

Untrusted Deserialization

untrusted-deserialization

STRIDE: Tampering

Description: When a technical asset accepts data in a specific serialized form (like Java or .NET serialization), Untrusted Deserialization risks might arise.

Detection: In-scope technical assets accepting serialization data formats (including EJB and RMI protocols).

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data assets processed and stored.

Wrong Communication Link Content

wrong-communication-link-content

STRIDE: Information Disclosure

Description: When a communication link is defined as readonly, but does not receive any data asset, or when it is defined as not readonly, but does not send any data asset, it is likely to be a model failure.

Detection: Communication links with inconsistent data assets being sent/received not matching their readonly flag or otherwise inconsistent protocols not matching the target technology type.

Rating: low

Wrong Trust Boundary Content

wrong-trust-boundary-content

STRIDE: Elevation of Privilege

Description: When a trust boundary of type network-policy-namespace-isolation contains non-container assets it is likely to be a model failure.

Detection: Trust boundaries which should only contain containers, but have different assets inside.

Rating: low

XML External Entity (XXE)

xml-external-entity

STRIDE: Information Disclosure

Description: When a technical asset accepts data in XML format, XML External Entity (XXE) risks might arise.

Detection: In-scope technical assets accepting XML data formats.

Rating: The risk rating depends on the sensitivity of the technical asset itself and of the data

assets processed and stored. Also for cloud-based environments the exploitation impact is at least medium, as cloud backend services can be attacked via SSRF (and XXE vulnerabilities are often also SSRF vulnerabilities).

Disclaimer

VaultNote Security Team conducted this threat analysis using the open-source Threagile toolkit on the applications and systems that were modeled as of this report's date. Information security threats are continually changing, with new vulnerabilities discovered on a daily basis, and no application can ever be 100% secure no matter how much threat modeling is conducted. It is recommended to execute threat modeling and also penetration testing on a regular basis (for example yearly) to ensure a high ongoing level of security and constantly check for new attack vectors.

This report cannot and does not protect against personal or business loss as the result of use of the applications or systems described. VaultNote Security Team and the Threagile toolkit offers no warranties, representations or legal certifications concerning the applications or systems it tests. All software includes defects: nothing in this document is intended to represent or warrant that threat modeling was complete and without error, nor does this document represent or warrant that the architecture analyzed is suitable to task, free of other defects than reported, fully compliant with any industry standards, or fully compatible with any operating system, hardware, or other application. Threat modeling tries to analyze the modeled architecture without having access to a real working system and thus cannot and does not test the implementation for defects and vulnerabilities. These kinds of checks would only be possible with a separate code review and penetration test against a working system and not via a threat model.

By using the resulting information you agree that VaultNote Security Team and the Threagile toolkit shall be held harmless in any event.

This report is confidential and intended for internal, confidential use by the client. The recipient is obligated to ensure the highly confidential contents are kept secret. The recipient assumes responsibility for further distribution of this document.

In this particular project, a timebox approach was used to define the analysis effort. This means that the author allotted a prearranged amount of time to identify and document threats. Because of this, there is no guarantee that all possible threats and risks are discovered. Furthermore, the analysis applies to a snapshot of the current state of the modeled architecture (based on the architecture information provided by the customer) at the examination time.

Report Distribution

Distribution of this report (in full or in part like diagrams or risk findings) requires that this disclaimer as well as the chapter about the Threagile toolkit and method used is kept intact as part of the distributed report or referenced from the distributed parts.